VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Submission 1

Smart Parking System for University Campus

INSTRUCTOR: Phan Trung Hieu
CLASS: CC02

Full Name	Student ID
QUACH GIA BAO	2452141
DIEC TUAN CHI	2452169
TO QUOC TAI	2453141
LY TRIEU PHONG	2452958
THOAI HAO	2452305
DOAN QUANG DANG	2352249
NGO QUANG HUNG	2352431
DANG TRAN THAI BAO	2452120

Ho Chi Minh City, March 1, 2026

Team Members & Task

Full Name	ID	Task	Percentage
QUACH GIA BAO	2452141	[PM & SC] Leader, PDF Composer, Module 2 (UC 2.4, 2.6, 2.7)	100%
DIEC TUAN CHI	2452169	[DEV] Non-interactive functional reqs, Module 1 (UC 1.5, 1.6, 1.7, 1.8)	100%
TO QUOC TAI	2453141	[Techlead & SysAdmin] Non-functional reqs, Module 1 (UC 1.1, 1.2, 1.3, 1.4)	100%
LY TRIEU PHONG	2452958	[DEV] Interactive functional reqs, Module 3	100%
THOAI HAO	2452305	[DEV] Non-interactive functional reqs, Module 2 (UC 2.1, 2.2, 2.3, 2.5)	100%
DOAN QUANG DANG	2352249	[QC] Non-functional reqs, Module 3	100%
NGO QUANG HUNG	2352431	[BA] Project details, Module 4	100%
DANG TRAN THAI BAO	2452120	[PO] Project details, Module 4	100%

Contents

1 Overview **3**
1.1 Project Description . . . 3
1.2 Stakeholders . . . 3
1.3 Objective & Scope . . . 4
1.3.1 Objective . . . 4
1.3.2 Scope . . . 5
1.4 User Stories . . . 6

2 Requirements **6**
2.1 Functional Requirements . . . 6
2.2 Non-Functional Requirements . . . 9

3 Use-case diagram & Use-case detail **11**
3.1 Module 1: Access Control . . . 11
3.2 Module 2: Monitoring & Guidance . . . 22
3.3 Module 3: Billing & Payment . . . 30
3.4 Module 4: System & Admin . . . 44

1 Overview

1.1 Project Description

The Smart Parking Management System (IoT-SPMS) is a technological solution designed to optimize vehicle parking activities for the community at Ho Chi Minh City University of Technology (HCMUT). The system serves a wide range of users, including undergraduate and graduate students, faculty members, staff, and external visitors. Currently, the increasing density of vehicles combined with limited parking capacity at the university has led to several significant challenges:

- Frequent congestion occurs at entry and exit points due to manual or fragmented processes.
- Users face difficulties finding available spots because there is no real-time visibility into parking occupancy.
- Inefficient management leads to poor utilization of existing parking spaces.
- Parking fee collection and billing are currently fragmented and lack synchronization

To solve these issues, our team is developing an IoT-based management platform that uses modern software engineering principles. Combining with occupancy sensors, local IoT gateway, and the university's existing infrastructure (such as HCMUT_SSO and BKPay), the system aims to automate access control when entry and exit, provide the parking slot guidance, billing system, ensure the stability of the system even during sensors malfunctions or bad connectivity. This system not only creates a more organized and modern campus environment but also significantly enhances the daily experience for all members of the HCMUT community.

1.2 Stakeholders

- Learners (undergraduate, graduate students, doctoral,...): entry/exit using official IDs, easy payment via BKPay.
- Faculty Members and Staffs (lecturers, officers,...): entry/exit using offical IDs, easy payment via BKPay.
- Temporary visitors (external guests without official ID): Obtain temporary ticket or card at the gate, also pay directly at the gate.

- Parking operator: manages parking area, real-time visibility of parking occupancy and tools to manage exceptions or sensors malfunction.
- System Administrators (Technical staff managing the system): configure user roles, pricing policies.
- HCMUT management : data consistency via HCMUT_DATACORE, traceability for financial.

1.3 Objective & Scope

1.3.1 Objective

The main goal of this project is to reduce the time and effort of the parking process for the end users by reducing congestion at entry and improving navigation flow, and for the operators by automating tedious and repetitive work. On the client's side, the system will aid with data consistency by integrating with existing systems and aid in clarity and accountability when digitally logging records.

The primary objectives of the project are:

- Operational:
  - Better utilization of available parking spaces.
  - Reduce congestion at hotspots through better navigation flow.
  - Reduce congestion at entrances by automating the entry process.
  - Reduce manual effort by automating guest's entry.
  - Increase system consistency by integrating with existing infrastructure (BKPay, HCMUT_SSO, etc.).
- User experience:
  - Faster and smoother check in/out process.
  - Guaranteed to find location of an available spot through the navigation system unless lot is full.
  - Better navigation when utilizing the signage system.
  - Guaranteed fair pricing through the fee policy.
  - Seemless payment with BKPay.
- Administrative:

  - Convenient and flexible system configuration.
  - Maintain complete, auditable logs of activities and transactions for compliance.
  - Clearer transparency and accountability by activity logging.
  - Streamlined exception and error control through access to log pool and central control hub.
  - Convenient interface for monitoring activities.

- Technical:
  - Guaranteed resilience with failsafe and fallback measures.
  - Able to handle high traffic at multiple entryways and intersection at the same time while integrating with other systems.
  - Deliver real-time occupancy updates from sensors to signage with minimal delay.
  - Prioritizing user's privacy and security through encryption and secure protocols.
  - Ensure smooth integration with existing systems such as HCMUT_SSO or BKPay.
  - Modular design allowing for easy debugging and addition of new systems.
  - Ensure the system scales well with a growing number of users and parking lots.

1.3.2 Scope

- Management of parking slot sensors, local IoT gateway
- User management
- System integration (integration with HCMUT_SSO)
- Exceptional handling

1.4 User Stories

- As a university member (Learner/Faculty/Staff), I want to use my official ID card to enter and exit the parking lot automatically.
- As a user, I want to see the real-time occupancy status (Available, Nearly Full, Full) on electronic signs so that I can decide where to park.
- As a Learner, I want the system to automatically track my parking sessions and calculate my fees at the end of the billing period.
- As a Learner, I want to pay my parking fees easily through the BKPay platform.
- As a Faculty or Staff member, I want to enjoy specific parking privileges and pricing according to my role and institutional regulations.
- As a Visitor or a user without an ID card, I want to be able to request a temporary access ticket at the entry gate to use the parking facility.
- As a Parking Operator, I want to monitor the status of all parking slots in near real-time to manage the facility efficiently.
- As a System Administrator, I want to configure the parking privileges and pricing policies for different user groups as regulations change.
- As a System Administrator, I want to access detailed logs of all parking activities and financial transactions for auditing and reporting purposes.
- As a System Administrator, I want to manage different user responsibilities through role-based access control.

2 Requirements

2.1 Functional Requirements

Interactive functional requirements includes:
* For Students and Faculty Members/Staff:

1. View real-time parking availability.
2. View personal parking and billing history.

3. Report payment issues.
4. View guiding signage.
5. Enter and exit parking with automatic authentication and ease of payment.

* For Temporary visitors:

1. Able to enter and exit parking without HCMUT IDs.
2. Report payment issues.
3. View guiding signage.
4. Rapid payment and easy to use.

* For Parking Operators:

1. Handle manual gateway control situations (When the system are down, or some functions are not working).
2. Resolve ID cards lost situations.
3. Notify commuters when the parking areas are full with no slot available.
4. View and manage parking availability.

* For Admin/System Operators:

1. Monitor real-time system statistics, user lists, and active parking sessions.
2. Configure policy settings and system settings.
3. Audit system logs for security and troubleshooting.
4. Manage roles.

* For Finance Office (Manage finance aspects of the parking zone):

1. View and export overall financial reports for a specified time period.
2. Configure billing settings and policies.

* For HCMUT Leaders and Managers:

1. View overall financial reports.

2. View parking system and usage reports.

Non-interactive functional requirements includes:

1. Automated User Data Synchronization.
   - The system will automatically synchronize personal and role related information from the university database. This must be done as a read-only manner to ensure data consistency and system security.
   - Frequency: daily
2. Periodic Billing.
   - For the university members group, the system will accumulate parking sessions over a predefined period. The system shall automatically compute the parking fees for each kind of users based on the policies and initiate payment requests using BKPAY.
   - Frequency: Periodically based on config.
3. Temporary Local Storage & Auto Sync.
   - If the central system goes down or network failure, the IoTs will automatically switch to offline mode, parking data will be saved into local storage. Once the problem is solved, the local data will be synchronized with the central database.
   - Frequency: Event-driven (Triggered by network disconnection and subsequent restoration).
4. Guidance Sign Adjustment.
   - The system continuously receives signals from parking sensors. When a vehicle parks or leaves, the system automatically calculates the available spots and updates the signage immediately.
   - Frequency: Event-driven (Triggered when a sensor's status changes).
5. Automated License Plate Recognition.
   - When a vehicle stops at the entry or exit gateway, the camera captures an image. The system then automatically runs an image processing algorithm to read the license plate text and saves it to the database to match the vehicle when it exits.

- Frequency: Event-driven (Triggered when a car arrives at the gateway).

Base on the functional requirements, we divide the system's use cases into 4 modules:

1. Module 1: Access control.
2. Module 2: Monitoring and guidance.
3. Module 3: Billing and Payment.
4. Module 4: System and Administration.

2.2 Non-Functional Requirements

Type 1: Performance requirements.

1. Gate access processing (RFID scan to barrier open) must complete in under 1.5 seconds to prevent congestion at entry and exit points.
2. The central system must support at least 500 concurrent requests per second without exceeding a 2-second response latency.
3. Financial reports, parking history lookups, system statistics must return results in under 5 seconds.
4. Payment through BKPAY API must be respond in less than 3 seconds.

Type 2: Reliability requirements.

1. The system must identify and flag disconnected sensors or gateways in 60 seconds of signal loss.
2. IoT gateways must be capable of local data buffering and basic access control for at least 24 hours during server down time.

Type 3: Security requirements

1. All university members must authenticate via HCMUT_SSO for Web App access.
2. Information synchronized from HCMUT_DATACORE must be handled in a read-only mode to prevent insecure modifications.

3. Role-Based Access Control (RBAC) must strictly isolate functions between actors.

4. All financial transactions, administrative actions must be logged and retained for a minimum of 12 months.

Type 4: Availability requirements

1. The system must maintain a minimum availability of 99.9% during the university's primary operational hours.

2. Electronic signages must update the parking status (Available, Nearly Full, Full) within 3 seconds of a state change detected by local sensors.

3. Downtime during operational hours must be less than 45 minutes per month, with no incident lasting longer than 10 minutes.

4. Scheduled maintenance and downtime during non-operational hours must be limited to a maximum of 10 hours per week and must be announced 24 hours in advance.

Type 5: Compatibility requirements

1. The system must provide automated billing requests compatible with the BKPAY API at the end of each configured period.

2. Data transmission between IoT devices and the central server must be consistent and utilize standard communication protocols.

3. User data must synchronize with HCMUT_DATACORE at least once every 24 hours to ensure the validity of parking privileges.

3 Use-case diagram & Use-case detail

3.1 Module 1: Access Control

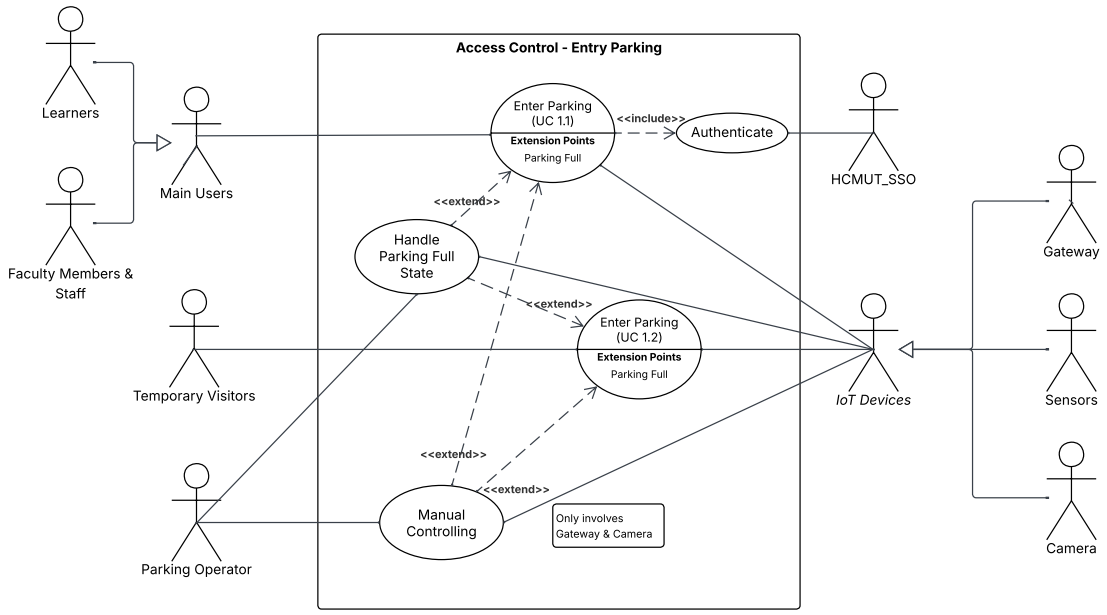

Figure 1: Use case diagram for Module 1: Enter scenario.

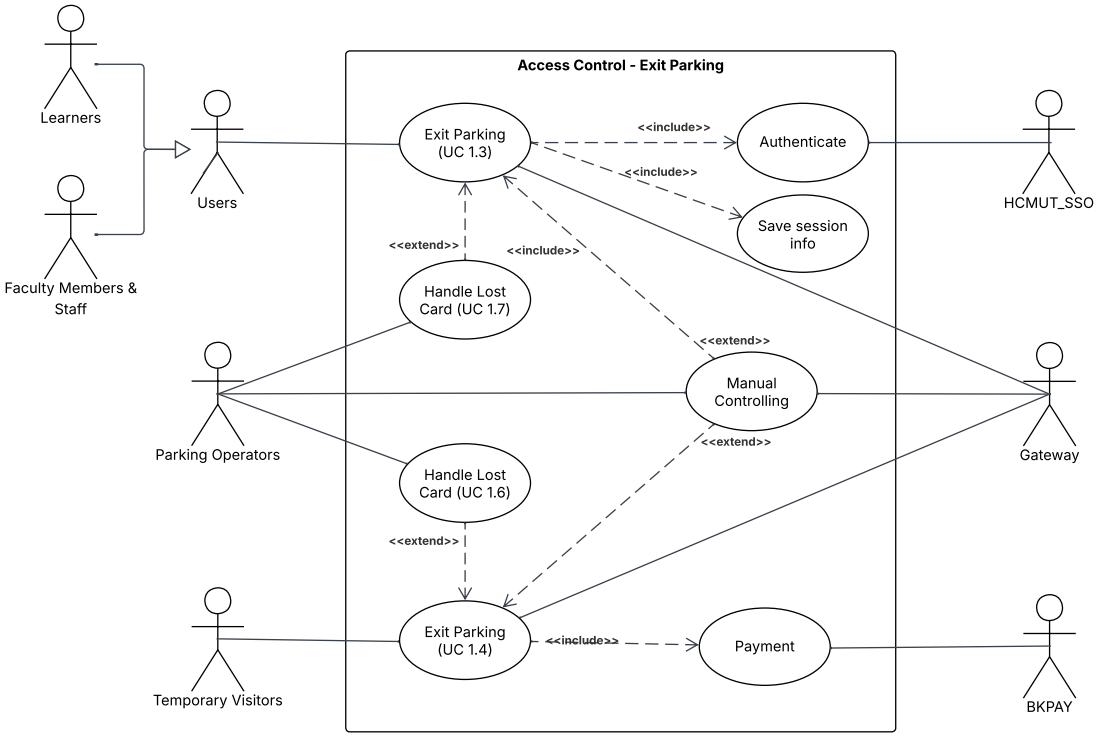

Figure 2: Use case diagram for Module 1: Exỉt scenario.

Table 1: UC 1.1. Enter Parking for Learners/Faculty Members

Field	**Details**
Use Case ID	1.1
Use Case Name	Enter Parking for Learners/Faculty Members
Actors	Primary: Learners, Faculty Members & Staff. Supporting: Gateway, Camera, Sensors, HCMUT_SSO.
Description	The automatic entry process for university members who possess a registered official ID card integrated with the SSO system.

Continued on next page

Table 1: UC 1.1. Enter Parking for Learners/Faculty Members (Continued)

Pre-conditions	The entry hardware is online. The barrier is closed.
Post-conditions	A new parking session is recorded. The barrier opens.
Main Flow	1. The user drives to the entry gate and taps their official ID card on the reader. 2. The camera captures the vehicle's license plate. 3. The system sends the card data to HCMUT_SSO for authentication. 4. Upon valid authentication, the system checks if the user's account is active and has no parking ban. 5. The system creates a new "Member Parking Session" containing the timestamp, card ID, and license plate. 6. The system opens the barrier. 7. Once the vehicle passes the loop detector, the barrier closes.
Alternate Flows	**E1: Parking Lot is Full:** If the system detects that the parking area has reached maximum capacity, the flow is transferred to UC 1.8 (Handle Parking Full State). The system will close the gateway and notify participants. The card tap is rejected. **E2: Sensor/Camera Error or Unrecognized Plate:** If the hardware fails to read the license plate or the automatic flow is interrupted, the system alerts the Parking Operator. The flow transfers to UC 1.5 (Manual Controlling). **E3: Forgot/Lost ID Card at Entry Gate:** If a member arrives without a card, the user presses the button on the card dispenser to take a temporary card. The process immediately transfers to UC 1.2 (Enter Parking for Temporary Visitors). **E4: Invalid/Inactive Card:** The SSO system returns an invalid status (e.g., account suspended, unpaid fees). The barrier remains closed, and an error message is displayed. The user can opt to take a temporary card (transfer to UC 1.2).

Table 2: UC 1.2: Enter Parking for Temporary Visitors

Field	Details
Use Case ID	1.2
Use Case Name	Enter Parking for Temporary Visitors
Actors	Primary: Temporary Visitors. Supporting: Gateway, Camera, Sensors.
Description	The entry process for guests who do not have a university ID, using a temporary card mechanism independent of the centralized SSO.
Pre-conditions	The card dispenser is functional and stocked with cards.
Post-conditions	An independent guest session is created. The barrier opens.
Main Flow	1. The visitor drives to the entry gate and presses the button on the card dispenser. 2. The dispenser issues a temporary parking card. 3. The camera captures the vehicle's license plate. 4. The system creates a new "Guest Parking Session" linked to the temporary card ID and the captured plate. 5. The system opens the barrier. 6. The barrier closes after the vehicle passes.
Alternate Flows	**E1: Parking Lot is Full:** The system transfers to UC 1.8 (Handle Parking Full State). The card dispenser is disabled, preventing new cards from being issued. **E2: Dispenser Empty or Hardware Failure:** If the card machine runs out of cards or malfunctions, the flow transfers to UC 1.5 (Manual Controlling) for the Parking Operator to manually process the entry.

Table 3: UC 1.3: Exit Parking for Learners/Faculty Members

Field	Details
Use Case ID	1.3
Use Case Name	Exit Parking for Learners/Faculty Members

Continued on next page

Table 3: UC 1.3: Exit Parking for Learners/Faculty Members (Continued)

Actors	Primary: Learners, Faculty Members & Staff. Supporting: Gateway, HCMUT_SSO.
Description	The exit process for registered members. The system verifies their entry session.
Pre-conditions	The user has an active parking session inside the campus.
Post-conditions	The parking session is marked as completed. The fee is calculated and added to their billing cycle.
Main Flow	1. The user drives to the exit gate and taps their official ID card. 2. The camera captures the license plate. 3. The system retrieves the active parking session associated with the card and verifies that the captured plate matches the entry plate. 4. The system calculates the fee (role-based pricing policy). 5. The system accumulates this fee into the user's current billing cycle (to be processed via BKPAY at the end of the month/cycle). 6. The system closes the session and opens the barrier. 7. The barrier closes after the vehicle passes.
Exception Flows	**E1: Lost ID Card inside the Campus:** If the member lost their ID card, the flow transfers to UC 1.7 (Main User Card Lost). The operator supports the member, invalidates the lost card, issues a replacement, and clears the session. **E2: Plate Mismatch or Barrier Stuck:** If the exit camera detects a mismatch or if the barrier malfunctions, the flow transfers to UC 1.5 (Manual Controlling) for manual verification. **E3: Unpaid Previous Debt:** The system detects an unpaid BKPAY bill. The barrier remains closed, prompting the user to settle their account.

Table 4: UC 1.4: Exit Parking for Temporary Visitors

Field	Details
Use Case ID	1.4
Use Case Name	Exit Parking for Temporary Visitors
Actors	Primary: Temporary Visitors. Supporting: Gateway, BKPAY.
Description	The exit process for guests, requiring a one-time immediate payment verification (via QR code transfer or cash) before exit.
Pre-conditions	The visitor has their temporary card.
Post-conditions	The fee is collected, the session is closed, and the vehicle exits.
Main Flow	1. The visitor hands the temporary card to the Guard or scans it at the exit terminal. 2. The camera captures the license plate to match with the entry record. 3. The system shows the parking fee. 4. The visitor pays the fee using cash or by scanning a banking QR (from BKPAY). 5. The Parking Operators confirms the receipt of payment in the system (or the system auto-detects the QR transfer). 6. The system closes the guest session and opens the barrier.
Exception Flows	**E1: Temporary Card Lost:** If the visitor loses their temporary card, the flow transfers to UC 1.6 (Temporary Card Lost). The operator verifies information, searches for the entry record via the license plate, invalidates the card, and calculates the fee/surcharge. **E2: Non-standard Exit (Payment Gateway Down or Barrier Error):** If the QR payment system is down or the barrier fails to lift, the flow transfers to UC 1.5 (Manual Controlling) for the Parking Operator to manually open the gate.

Table 5: UC 1.5: Manual Controlling

Field	Details
Use Case ID	1.5
Use Case Name	Manual Controlling
Actors	Primary: Parking Operators. Supporting: Gateway.
Description	Allows the Parking Operator to manually control the barrier gate via the software interface during emergencies, sensor failures, or non-standard entry/exit exceptions.
Pre-conditions	A hardware failure, unreadable plate, or emergency has occurred (Flow transferred from 1.1, 1.2, 1.3, or 1.4). The operator is logged into the system dashboard.
Post-conditions	The barrier state is manually changed (opened/closed), and the manual override event is securely logged for auditing.
Main Flow	1. The system alerts the Operator of a manual intervention request (e.g., license plate mismatch, hardware failure). 2. The Operator verifies the physical situation (e.g., visually checks the vehicle's actual license plate or identifies an emergency vehicle). 3. The Operator inputs the necessary missing data into the system (e.g., manual plate entry) or selects an override reason from a predefined list (e.g., "Emergency", "System Error"). 4. The Operator clicks the manual override button on the system dashboard. 5. The system logs the manual override event, including the timestamp, Operator ID, and selected reason. 6. The system sends an execution command to the Gateway. 7. The Gateway physically triggers the barrier to open, allowing the vehicle to pass.

Continued on next page

Table 5: UC 1.5: Manual Controlling (Continued)

Exception Flows	**E1: Gateway Connection Lost:** The software cannot communicate with the physical Gateway due to a network or power failure. The system displays a connection error. The Operator must leave the booth and use a physical mechanical key to lift the barrier.

Table 6: UC 1.6: Temporary Card Lost

Field	**Details**
Use Case ID	1.6
Use Case Name	Temporary Card Lost
Actors	Primary: Parking Operators. Supporting: Temporary Visitors.
Description	Handles the scenario where a temporary visitor loses their parking card before exiting. The system calculates the fee and invalidates the lost card to prevent unauthorized use.
Pre-conditions	The visitor arrives at the exit gate without their temporary card. The flow is transferred from UC 1.4.
Post-conditions	The lost card is temporarily invalidated, the penalty/parking fee is collected, and the guest session is closed.

Continued on next page

Table 6: UC 1.6: Temporary Card Lost (Continued)

Main Flow	1. The Temporary Visitor informs the Parking Operator about the lost card. 2. The Operator uses the system's search function, entering the vehicle's license plate captured by the exit camera. 3. The system retrieves the original "Guest Parking Session" matching the license plate and displays the entry time. 4. The Operator verifies the vehicle and driver identity. 5. The system calculates the standard parking fee based on duration, plus an automatic "Lost card Surcharge." 6. The Operator collects the total fee via cash or QR code. 7. The system temporarily marks the lost temporary card ID as "Invalid" to prevent it from being used by someone else later. 8. The system closes the session and triggers the barrier to open for exit.
Exception Flows	**E1: License Plate Not Found:** The system cannot find an entry record matching the license plate. The Operator must manually search the database by estimated entry time or vehicle description to locate the session.

Table 7: UC 1.7: Main User Card Lost

Field	**Details**
Use Case ID	1.7
Use Case Name	Main User Card Lost
Actors	Primary: Parking Operators. Supporting: Learners, Faculty Members & Staff.
Description	Manages the situation where a university member loses their official ID card while parked inside the campus. The operator assists in verifying their identity, invalidating the old card, and clearing the session.

Continued on next page

Table 7: UC 1.7: Main User Card Lost (Continued)

Pre-conditions	A member arrives at the exit without their ID card. The flow is transferred from UC 1.3.
Post-conditions	The lost ID card is deactivated in the parking system, the fee is linked to BKPAY, and the user is allowed to exit.
Main Flow	1. The university member notifies the Parking Operator that they lost their ID card inside the campus. 2. The Operator looks up the member's active parking session using their vehicle's license plate or their student/staff ID number. 3. The Operator verifies the member's identity against the system data (read-only data synced from HCMUT_DATACORE, such as checking their profile picture). 4. The system calculates the parking fee and adds it to the user's BKPAY billing cycle as usual. 5. The Operator marks the lost ID card as "Invalid/Lost" in the system to prevent unauthorized access if someone else finds it. 6. The system closes the active parking session. 7. The Operator manually triggers the barrier to let the member exit.
Exception Flows	**E1: Identity Verification Fails:** The driver does not match the profile picture or information associated with the student/staff ID in the system. The Operator refuses exit and contacts campus security for suspected vehicle theft.

Table 8: UC 1.8: Handle Parking Full State

Field	**Details**
Use Case ID	1.8
Use Case Name	Handle Parking Full State

Continued on next page

Table 8: UC 1.8: Handle Parking Full State (Continued)

Actors	Primary: Parking Operators. Supporting: Sensors, Gateway.
Description	Triggers when the parking lot reaches maximum capacity. The system automatically updates signage, halts new entries, and alerts the operator.
Pre-conditions	The system's capacity counter (based on IoT sensors and active sessions) reaches the predefined maximum limit. Flow transferred from UC 1.1 or 1.2.
Post-conditions	The entry gateway is temporarily closed to new incoming traffic, and users are notified.
Main Flow	1. The IoT Sensors and system database confirm that all parking slots are currently occupied. 2. The system automatically updates the external electronic signage to display "FULL". 3. The system sends a command to the entry Gateway to disable the card dispenser and reject incoming card taps. 4. The system sends an alert to the Parking Operator's dashboard indicating the lot has reached full capacity. 5. Participants arriving at the gate are notified by the signage (or the operator) to use alternative parking zones. 6. The state remains active until a vehicle exits, freeing up a space, at which point the system automatically reverts the gateway and signage to the "Available" state.
Exception Flows	**E1: Sensor Miscalibration (False Full State):** The system reports "FULL," but the Operator sees empty spaces physically available. The Operator uses a manual override function to correct the slot count and forcefully re-enable the entry gateway.

3.2 Module 2: Monitoring & Guidance

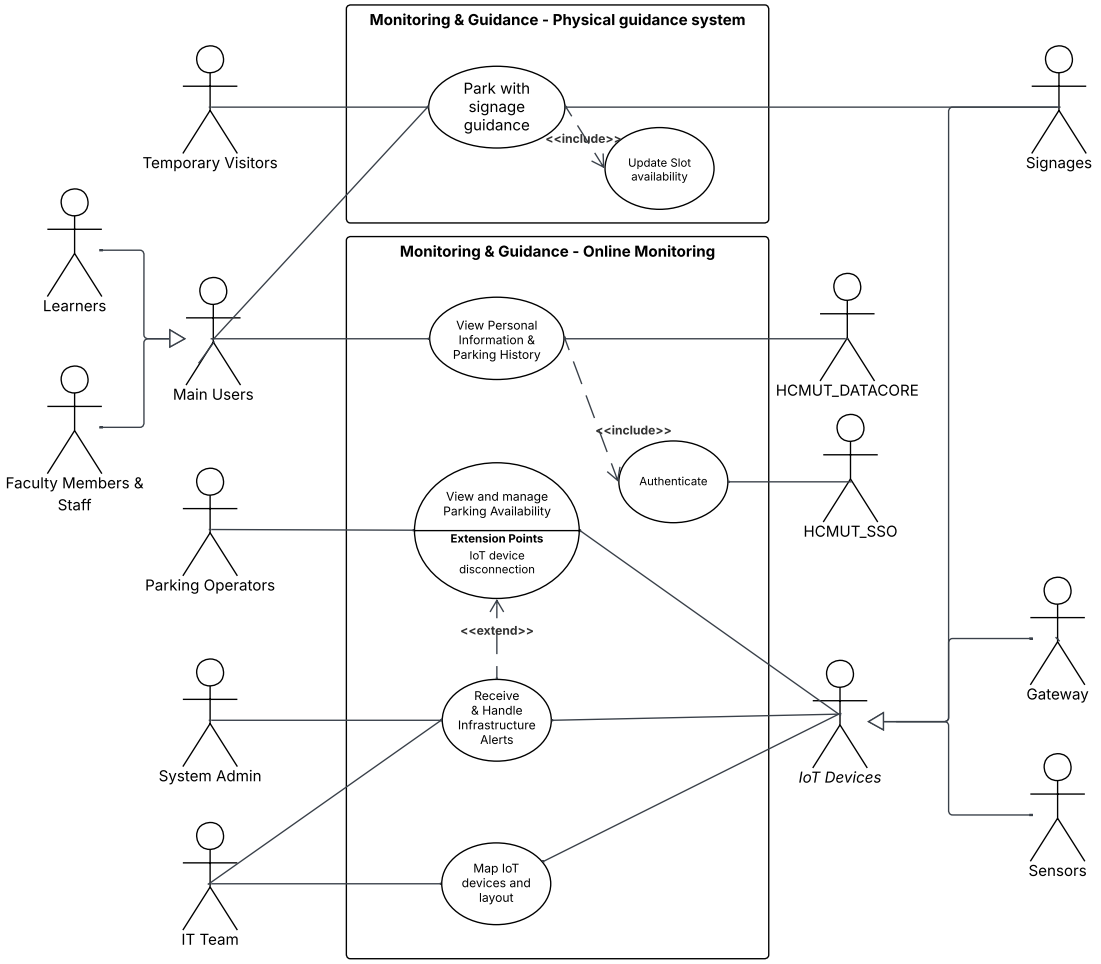

Figure 3: Use case diagram for Module 2.

Table 9: UC 2.1: Realtime park guidance for commuters

Field	Details

Continued on next page

Table 9: UC 2.1: Realtime park guidance for commuters (Continued)

Use Case ID	2.1
Use Case Name	Realtime park guidance for commuters
Actors	Primary: Learners, Faculty Member & Staff, Temporary Visitors. Supporting: Signages.
Description	Main users and Temporary visitors get guidance via the parking signages. The available slots for each way/area is displayed on the signages.
Trigger	A commuter arrives at the campus parking entrances or drives past key intersections.
Pre-conditions	IoT parking sensors and local gateways are online. Signages are currently working.
Post-conditions	The commuter successfully parks in an available slot, and the system updates the real-time availability on its signage.
Main Flow	1. The commuter arrives at the entrance of the parking lot. 2. The commuter observes signage at the entrance which displays availability (full, almost full, available). 3. The commuter enters the parking facility. 4. At intersections, electronic signage provides directional information and available slots. 5. The commuter finds the parking spot. 6. IoT sensor detects new occupancy and relays information through local gateways; the system updates signages and maps.

Continued on next page

Table 9: UC 2.1: Realtime park guidance for commuters (Continued)

Alternate Flows	**A1: Full Facility (at step 2):** Signage displays zero slots available. Gateway declines to open (UC 1.8). The commuters are redirected to other parking zones of HCMUT. **A2: Zone Full En Route (at step 4):** While driving towards Zone A, other vehicles occupy the remaining spots. The system detects the change, recalculates the nearest available zone (e.g., Zone B), and updates the Directional Signage to redirect the commuter. The commuter follows the updated directions and resumes Step 5.
Exception Flows	**E1: Hardware/Connection Failure:** The system detects a loss of connection or data timeout. If sensors fail, the system excludes those slots and resyncs data. If the entire system fails, signages fallback to error mode, guiding commuters to follow parking operators' instructions. The system triggers an alert to the IT Team via the Web App (UC 2.4).

Table 10: UC 2.2: View and manage parking availability

Field	**Details**
Use Case ID	2.2
Use Case Name	View and manage parking availability
Actors	Primary: Parking Operators. Supporting actors: Gateways, Sensors, Signages.
Description	Monitor security, coordinate the parking lot, and inspect detailed slot data (time in/out, duration) and specific errors.
Trigger	Parking operator accesses the monitor or an error occurs.
Pre-conditions	1. The parking operator is authenticated via UC 4.1. 2. The system has an active connection to the IoT network.
Post-conditions	1. Operators maintain the facility. 2. Any detected errors are flagged for maintenance.

Continued on next page

Table 10: UC 2.2: View and manage parking availability (Continued)

Main Flow	1. The Parking Operators are successfully logged into the Web Management System. 2. The operator accesses the "Real-time Occupancy Map" on the Web App. 3. The system retrieves data via connections with sensors, signages and gateway to display real-time slot maps and individual slot details. 4. The operator selects a specific slot to view detailed metadata (e.g., Plate ID, Entry Time, Current Duration). 5. The operator monitors high-traffic zones to coordinate manual guidance if necessary.
Alternate Flows	**A1: Search with filter for specific data (at step 4):** The operator uses the filter tool to view only overstayed vehicles (e.g., cars parked >24 hours). The system highlights the filtered slots on the map, and the operator exports the list for security verification.
Exception Flows	**A1: Sensors disconnection (at step 3):** The system detects lost connection or timeout from a specific IoT device. The status is updated to "OFFLINE/UNKNOWN". The system and/or Operators notify the IT team and System Admin. The system generates a maintenance ticket in the Error log.

Table 11: UC 2.3: Map IoT devices and layout

Field	**Details**
Use Case ID	2.3
Use Case Name	Map IoT devices and layout
Actors	Primary: IT Team. Supporting: Gateway, Sensors, Signages.

Continued on next page

Table 11: UC 2.3: Map IoT devices and layout (Continued)

Description	The IT Team opens a mapping session by accessing the "Device Configuration & Layout Mapping" interface on the Web App.
Trigger	New sensors are installed, replaced, or the layout of the parking lot changes.
Pre-conditions	1. The IT Team is authenticated via UC 4.1. 2. IoT devices are powered on and connected to the network. 3. The digital layout/blueprint of the parking lot has been uploaded to the system.
Post-conditions	IoT device IDs are successfully linked to physical slot coordinates in the database.
Main Flow	1. IT Team accesses the "Layout Mapping & Configuration" interface. 2. System retrieves and lists all detected but unassigned Device IDs. 3. The IT Team selects a specific Device ID on the list. 4. The IT Team identifies and clicks on the corresponding physical slot or signage location on the digital map. 5. The system validates the pairing, saves the mapping to the database, and sends signals to test connection. 6. The IT Team verifies the changes.

Continued on next page

Table 11: UC 2.3: Map IoT devices and layout (Continued)

Alternate Flows	**A1: Device replacement (at step 1):** The IT Team selects a slot with an assigned Device ID on the digital map. The IT Team chooses the "Replace Device" option with an unassigned Device ID from the unassigned list. The system unbinds the old ID and assigns the new ID to the slot coordinates. Resumes at step 5. **A2: Signage mapping (at step 3):** The IT Team selects a Device ID with "Signage" type and clicks on the intersection/entrance location on the map. The system prompts the IT Team to select which "Zones" or "Areas" this signage should display data for. The IT Team confirms the selection and the system pushes the configuration. Resumes at step 5.
Exception Flows	**E1: New device ID Not Detected (at step 2):** A physically installed sensor does not appear in the "Unassigned" list. The IT Team must physically troubleshoot the problem by either replacing the device or manually resetting it. The IT Team clicks refresh list and resumes at step 3.

Table 12: UC 2.4: Receive & Handle Infrastructure Alerts

Field	**Details**
Use Case ID	2.4
Use Case Name	Receive & Handle Infrastructure Alerts
Actors	Primary: Parking Operators, IT Team, System Admin. Supporting: Gateway, Sensors, Signages.
Description	The system monitors the health of all hardware components. When a failure is detected, an alert is pushed to the Web App for the management team to identify and resolve.
Trigger	The system detects a hardware issue from the IoT devices.

Continued on next page

Table 12: UC 2.4: Receive & Handle Infrastructure Alerts (Continued)

Pre-conditions	1. Actors are logged into the Web App with active notification permissions. 2. All IoT devices have been successfully registered and mapped. 3. The system monitoring service is active.
Post-conditions	1. The failed device is identified and logged. 2. Maintenance individuals are notified for physical inspection.
Main Flow	1. The system continuously monitors signals from all IoT devices. 2. The system detects a device failure, triggers an alert on the web app for all related actors, and flags the device location. 3. Actors click on the alert notification to view details. 4. The system displays the exact location, Device ID, and error type on the digital map. 5. The IT Team acknowledges the alert and dispatches personnel for on-site troubleshooting. 6. The system records the incident in the system log.
Alternate Flows	**A1: Remote device reboot (at step 5):** If the device is rebootable, the IT Team attempts a remote reboot via the Web App. If the device comes back online, the system clears the alert automatically and returns the status to normal. If the reboot fails, the flow resumes at step 5 (dispatching personnel).
Exception Flows	None.

Table 13: UC 2.5: View Personal Information & Parking History

Field	**Details**
Use Case ID	2.5
Use Case Name	View Personal Information & Parking History

Continued on next page

Table 13: UC 2.5: View Personal Information & Parking History (Continued)

Actors	Primary: Learners, Faculty Members & Staff. Supporting: HCMUT_DATACORE.
Description	Allow users to view their parking sessions (time in/out, license plate, calculated fees) and personal info on the WEB APP via a read-only API from HCMUT_DATACORE.
Trigger	User navigates to their personal profile/parking history page on the WEB APP.
Pre-conditions	User is authenticated via HCMUT_SSO (UC 4.1).
Post-conditions	User successfully views their data, and accurate fees are displayed based on logged sessions.
Main Flow	1. The user logs into the WEB APP. 2. The system calls a read-only API to HCMUT_DATACORE for information. 3. The system retrieves the list of parking sessions from the database. 4. The system calculates total unpaid fees from previous sessions. 5. The system displays the combined personal information and parking history.
Alternate Flows	None.
Exception Flows	**E1: HCMUT_DATACORE unresponsive:** The system cannot communicate with the server (bad gateway/error). The app displays cached history and an error message: "Personal profile information currently unavailable." The IT Team is alerted (UC 2.4). **E2: Local IoT gateway problem:** If data from sensors is lost, the WEB APP displays "Active parking currently unavailable" and triggers an alert to the IT Team (UC 2.4).

3.3 Module 3: Billing & Payment

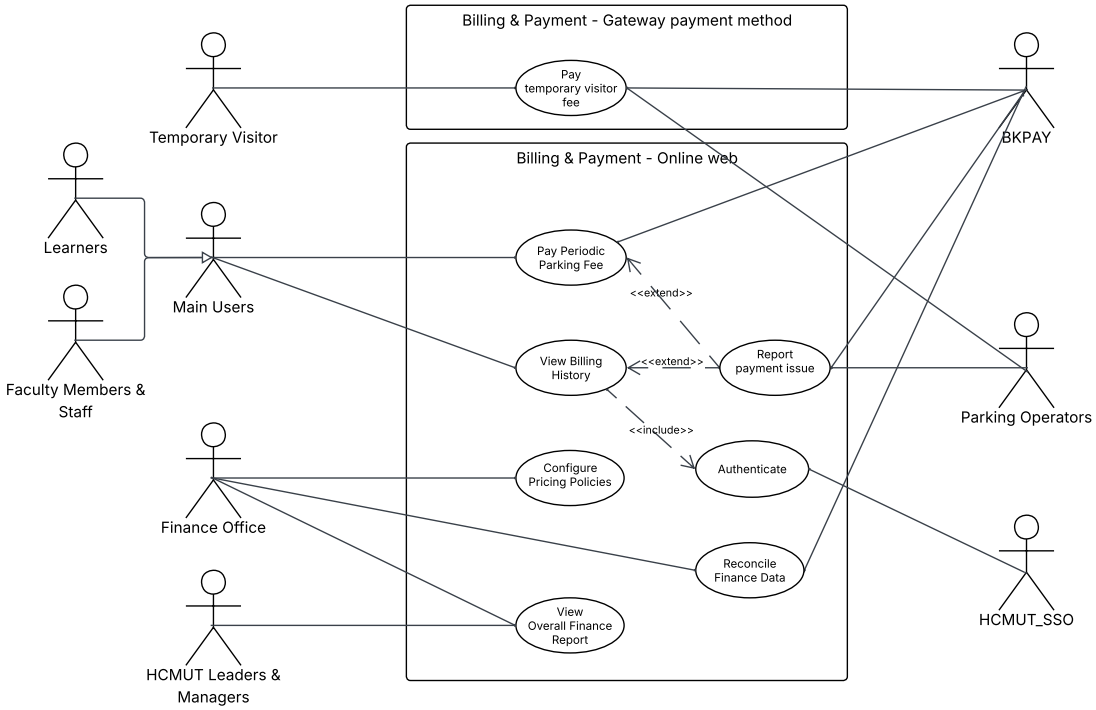

Figure 4: Use case diagram for Module 3.

Table 14: UC 3.1: Pay periodic parking fee

Field	Details
Use Case ID	3.1
Use Case Name	Pay periodic parking fee
Actors	Primary: Learners, Faculty Members & Staff. Supporting: BKPAY.

Continued on next page

Table 14: UC 3.1: Pay periodic parking fee (Continued)

Description	This use case allows registered users to pay their periodic parking fees via the BKPAY system using a linked OCB account or a dynamic QR code.
Trigger	The user navigates to the "Billing & Payment" section on the web app and clicks "Pay Fee."
Pre-conditions	1. User is authenticated via HCMUT_SSO. 2. Profile is synchronized from HCMUT_DATACORE. 3. Outstanding fee is calculated based on institutional pricing policy.
Post-conditions	1. Fee is successfully paid and balance is cleared. 2. Transaction is saved in the Billing History.
Main Flow	1. The user accesses the "Billing & Payment" module. 2. The system retrieves and displays the current outstanding periodic parking fee. 3. The user selects the fee and clicks "Pay Now." 4. The system initiates the request and redirects the user to BKPAY. 5. BKPAY displays options: Sub-flow A (OCB) or Sub-flow B (QR Code). 6. BKPAY sends a success confirmation back to the parking system. 7. The system updates the fee status to "Paid" and clears the balance. 8. The system saves transaction details and redirects to a success page.
Sub-flows	**Sub-flow A: Linked OCB account** The user selects OCB method. BKPAY displays fee details. The user confirms, and the system automatically deducts the amount from the linked account. **Sub-flow B: Dynamic QR Code** The user selects QR method. BKPAY generates a VietQR code. The user scans via banking app and authorizes the transfer. BKPAY receives the notification.

Continued on next page

Table 14: UC 3.1: Pay periodic parking fee (Continued)

Alternate Flows	**A1 - User cancels payment:** At step 5, the user chooses to cancel. BKPAY redirects back with a "Cancelled" status. The system displays a cancellation message and the fee remains unpaid.
Exception Flows	**E1 - Payment failed/Declined:** BKPAY declines the transaction (insufficient funds, wrong OTP). The system displays: "Payment failed. Please check details or try another method." **E2 - BKPAY Gateway Timeout:** The system fails to connect to BKPAY. The system logs the error and displays: "Payment gateway is currently unavailable. Please try again later."

Table 15: UC 3.2: Pay temporary visitor fee

Field	**Details**
Use Case ID	3.2
Use Case Name	Pay temporary visitor fee
Actors	Primary: Temporary Visitors. Supporting: Parking Operators, BKPAY (Payment Gateway).
Description	This use case allows temporary visitors to pay their parking fees when exiting the campus via a dynamic BKPAY QR code or cash.
Trigger	The Temporary Visitor arrives at the exit gate and presents their temporary parking card.
Pre-conditions	1. Valid ongoing parking session exists. 2. Visitor is physically present at the exit gate. 3. Fee is calculated based on duration and visitor pricing policy.

Continued on next page

Table 15: UC 3.2: Pay temporary visitor fee (Continued)

Post-conditions	1. Fee is successfully paid and session is closed. 2. Transaction is saved in the database. 3. Exit barrier opens (Triggers UC 1.4).
Main Flow	1. The Temporary Visitor presents the temporary parking card at the exit gate. 2. The Parking Operator scans the card (or the visitor scans it at the automated terminal). 3. The system retrieves the session, calculates the fee, and displays the total amount. 4. The visitor selects a payment method: Sub-flow A (QR Code) or Sub-flow B (Cash). 5. Upon successful payment, the system updates the status to "Paid". 6. The system saves the transaction details into the database. 7. The system displays a success message and triggers the barrier to open.
Sub-flows	**Sub-flow A: Pay via Dynamic QR Code (BKPAY)** The visitor chooses bank transfer. The system displays a dynamic VietQR code. The visitor scans via a mobile banking app and authorizes the transfer. BKPAY sends a confirmation to the parking system. **Sub-flow B: Pay via Cash** The visitor hands cash to the Parking Operator. The Operator returns change if necessary and manually confirms the receipt by clicking "Confirm Cash Payment" on the system interface.
Alternate Flows	**A1 - Change Payment Method:** At step 4, the visitor decides to switch methods (e.g., from QR to Cash). The Operator cancels the QR generation and selects the Cash option, proceeding to Sub-flow B.

Continued on next page

Table 15: UC 3.2: Pay temporary visitor fee (Continued)

Exception Flows	**E1 - Card Lost/Unreadable:** The system cannot automatically calculate the fee. The flow transfers to UC 1.6 (Handle temporary card lost) for manual penalty processing. **E2 - BKPAY Webhook Delay/Network Issue:** If the transfer is successful on the visitor's phone but the system doesn't receive the webhook, the Operator visually verifies the transaction and uses the "Manually Override - Force Paid" function with a recorded note.

Table 16: UC 3.3: Configure Pricing Policy

Field	**Details**
Use Case ID	3.3
Use Case Name	Configure Pricing Policy
Actors	Primary: HCMUT Finance Office. Supporting: None.
Description	This use case allows authorized staff from the HCMUT Finance Office to manage, configure, and update the dynamic pricing policies for different user categories and vehicle types.
Trigger	The Finance Officer navigates to the "Pricing Configuration" module on the web app and clicks "Create" or "Edit" on a pricing policy.
Pre-conditions	1. The user has successfully logged into the web application via HCMUT_SSO. 2. The user's role is strictly verified via RBAC as authorized for financial configurations (e.g., Finance Admin).
Post-conditions	1. The new or updated pricing policy is successfully saved. 2. The system applies the new policy for all subsequent calculations. 3. The action is recorded in the system's Audit Trail.

Continued on next page

Table 16: UC 3.3: Configure Pricing Policy (Continued)

Main Flow	1. The user accesses the "Pricing Configuration" module on the web application. 2. The system retrieves and displays current pricing policies categorized by user roles and vehicle types. 3. The user selects an existing policy to modify or clicks "Create New Policy". 4. The system displays the policy configuration form. 5. The user inputs or adjusts pricing parameters (base fee, billing cycle, free quotas, block-rate pricing, etc.). 6. The user clicks the "Save Policy" button. 7. The system validates the input parameters (checking for negative values, overlapping blocks, etc.). 8. The system updates the pricing policy records in the database. 9. The system logs the event into the Audit Trail (User ID, timestamp, before/after values). 10. The system redirects to the list view and displays a success message.
Alternate Flows	**A1 - Cancel Configuration:** At step 5, the user decides not to proceed and clicks "Cancel". The system discards all unsaved changes and redirects to the policy list view.
Exception Flows	**E1 - Invalid Input Parameters:** At step 7, the system detects logical errors. It prevents saving, highlights erroneous fields in red, and displays: "Invalid configuration detected. Please correct the highlighted fields and try again." **E2 - Unauthorized Access Attempt:** If a user without the "Finance Admin" role attempts to access this module, the system denies access immediately, logs the security warning, and redirects the user with an "Access Denied" message.

Table 17: UC 3.4: View Billing History

Field	Details
Use Case ID	3.4
Use Case Name	View Billing History
Actors	Primary: Learners, Faculty Members & Staff. Supporting: None.
Description	This use case allows authenticated university members to review their past parking sessions, accumulated fees, and payment statuses (Paid, Unpaid, Cancelled) over specified billing cycles.
Trigger	The user navigates to the "Billing History" or "My Parking" section on the web application.
Pre-conditions	1. The user has successfully authenticated and logged in via HCMUT_SSO. 2. The user's profile and role information are correctly synchronized from HCMUT_DATACORE.
Post-conditions	1. The system successfully retrieves and displays the user's historical billing and parking data. 2. No data in the system is modified (Read-only operation).

Continued on next page

Table 17: UC 3.4: View Billing History (Continued)

Main Flow	1. The user selects the "Billing History" tab on their personal dashboard. 2. The system retrieves the user's parking sessions and transaction records from the database, defaulting to the current or most recent billing cycle. 3. The system displays a chronological list of parking sessions including: Entry/Exit Date & Time, License Plate, Total Duration, Applied Policy, Fee Amount, and Payment Status. 4. The user uses filter options to select a specific time range or sort by Payment Status. 5. The system dynamically updates the displayed list based on the user's selected filters. 6. The user clicks on a specific "Paid" transaction to view its details. 7. The system displays an electronic receipt detailing the Transaction ID, Date of Payment, and the Payment Method used.
Alternate Flows	**A1 - Export Billing History:** At step 5, the user clicks the "Export" button. The system generates and downloads a formatted PDF or Excel report of the currently filtered billing history. **A2 - No Records Found:** At step 2 or 5, if no activity is found, the system displays an empty state with the message: "No parking history found for the selected period."
Exception Flows	**E1 - Database Connection Timeout / Retrieval Error:** At step 2 or 5, the system fails to connect or the query times out. The system safely aborts the loading process and displays: "Unable to retrieve billing history at this time. Please check your connection or try refreshing the page."

Table 18: UC 3.5: View Overall Finance Report

Field	Details
Use Case ID	3.5
Use Case Name	View Overall Finance Report
Actors	Primary: HCMUT Finance Office, HCMUT Leaders & Managers. Supporting: None.
Description	This use case allows authorized financial staff and university management to view, analyze, and export aggregated financial data from the smart parking system through strategic dashboards and detailed reports.
Trigger	The user navigates to the "Financial Reports" or "Management Dashboard" module on the web application.
Pre-conditions	1. The user has successfully authenticated and logged in via HCMUT_SSO. 2. The user's role is strictly verified via RBAC as either "Finance Admin" or "University Management."
Post-conditions	The system successfully presents the aggregated financial data without modifying any underlying transaction records (Read-only operation).

Continued on next page

Table 18: UC 3.5: View Overall Finance Report (Continued)

Main Flow	1. The user selects the "Financial Reports" module from the navigation menu. 2. The system queries the database to aggregate revenue data across all parking sessions and payment methods (BKPAY periodic, BKPAY dynamic QR, and Cash). 3. The system displays a high-level strategic dashboard including: Total revenue, revenue breakdown by user categories, and visual charts showing trends over time. 4. The user applies data filters to adjust the strategic view (e.g., selecting a specific quarter, year, or a specific parking zone). 5. The system dynamically updates the dashboard metrics and charts based on the applied filters. 6. The user clicks the "Export Financial Report" button to download the data. 7. The system generates a formal report (PDF or Excel) containing both the high-level summary and detailed transaction logs. 8. The system initiates the file download to the user's device.
Alternate Flows	**A1 - No Financial Data for Selected Period:** At step 5, if no transactions match the selected filters, the system displays empty charts and a message: "No financial data available for the selected period."
Exception Flows	**E1 - Data Retrieval Timeout (Large Data Volume):** At step 2 or 5, if a query causes a timeout, the system safely aborts the process and displays: "The requested report is too large and timed out. Please select a narrower time range (e.g., up to 6 months) and try again."

Table 19: UC 3.6: Report payment issue

Field	**Details**
Use Case ID	3.6

Continued on next page

Table 19: UC 3.6: Report payment issue (Continued)

Use Case Name	Report payment issue
Actors	Primary: Learners, Faculty Members & Staff. Supporting: HCMUT Finance Office, BKPAY, Parking Operators.
Description	This use case allows registered users to submit a formal dispute or report an error regarding their parking fees. The system captures the issue and routes it as a support ticket for investigation and resolution.
Trigger	The user navigates to the "Billing History" module and clicks the "Report Issue" button next to a specific parking session or payment transaction.
Pre-conditions	1. The user has successfully authenticated and logged in via HCMUT_SSO. 2. The user has an existing parking session or transaction record in the system to report against.
Post-conditions	1. A support ticket is successfully created with a "Pending Review" status. 2. The disputed transaction is flagged as "Under Investigation". 3. The HCMUT Finance Office (and relevant support teams) is notified.

Continued on next page

Table 19: UC 3.6: Report payment issue (Continued)

Main Flow	1. The user views their transaction list in the "Billing History" module. 2. The user identifies a problematic transaction and clicks the "Report Issue" button associated with it. 3. The system displays a "Dispute/Report Issue" form pre-filled with the Transaction ID, Date, and relevant fee details. 4. The user selects an "Issue Category" from a dropdown menu (e.g., "Overcharged", "Bank Deducted but Status Unpaid"). 5. The user types a detailed description of the problem in the text area. 6. The user uploads supporting evidence (e.g., a screenshot of the banking app deduction receipt). 7. The user clicks "Submit Report". 8. The system validates the form data (checking mandatory fields and file constraints). 9. The system creates a support ticket, links it to the original transaction, and updates the transaction state to "Under Investigation". 10. The system routes the ticket to the HCMUT Finance Office dashboard. 11. The system displays a success message containing the Ticket ID and estimated resolution time, and sends a confirmation email.
Alternate Flows	**A1 - Cancel Report:** At step 5 or 6, the user decides not to proceed and clicks "Cancel". The system closes the form, discards all inputs, and returns the user to the Billing History view.

Continued on next page

Table 19: UC 3.6: Report payment issue (Continued)

Exception Flows	**E1 - Invalid File Upload:** At step 8, if the uploaded screenshot exceeds the limit (>5MB) or is in an unsupported format, the system rejects the submission and displays: "File size must be under 5MB and in JPG, PNG, or PDF format. Please re-upload your proof." **E2 - Missing Mandatory Information:** At step 8, if the user leaves the Description or Issue Category blank, the system prevents submission and highlights the missing fields in red.

Table 20: UC 3.7: Reconcile Financial Data

Field	**Details**
Use Case ID	3.7
Use Case Name	Reconcile Financial Data
Actors	Primary: HCMUT Finance Office. Supporting: BKPAY.
Description	This use case allows the Finance Office to cross-check records in the Smart Parking System against BKPAY settlement data to identify and resolve financial discrepancies.
Trigger	The Finance Officer initiates a manual sync via the "Reconciliation" module, or the system triggers an automated background reconciliation.
Pre-conditions	1. User is authenticated via HCMUT_SSO. 2. User possesses the "Finance Admin" role. 3. BKPAY APIs or settlement files are accessible.
Post-conditions	1. Discrepancies are identified and resolved. 2. Relevant support tickets from UC 3.6 are updated. 3. Reconciliation report is logged in the Audit Trail.

Continued on next page

Table 20: UC 3.7: Reconcile Financial Data (Continued)

Main Flow	1. The user accesses the "Reconciliation" module on the web application. 2. The user selects a date range and clicks "Run Reconciliation." 3. The system retrieves all local parking transaction records for the period. 4. The system calls the BKPAY API (or parses an uploaded file) to fetch actual payment logs. 5. The system performs an automated cross-match based on Transaction IDs and amounts. 6. The system displays a Dashboard with three tabs: Matched, Discrepancies, and Pending Tickets. 7. The user reviews a mismatched transaction in the "Discrepancies" tab. 8. The system highlights the difference (e.g., Local = Unpaid, BKPAY = Paid). 9. The user clicks "Force Sync - Mark as Paid." 10. The system updates the local status to "Paid," clears debt, and logs the manual override. 11. If linked to a UC 3.6 ticket, the system marks it as "Resolved" and notifies the user.
Alternate Flows	**A1 - 100% Match (No Discrepancies):** If all records match, the system displays a "Perfect Match" green banner. The user clicks "Approve & Close Cycle." **A2 - Initiate Refund:** If an overcharge is detected, the Finance Officer clicks "Initiate Refund." The system sends a request to BKPAY and sets the status to "Refund Pending."
Exception Flows	**E1 - BKPAY API Connection Failure:** The system fails to connect or the API key is expired. The system halts the process and displays: "Unable to connect to BKPAY settlement server. Please check configuration or upload a manual file."

3.4 Module 4: System & Admin

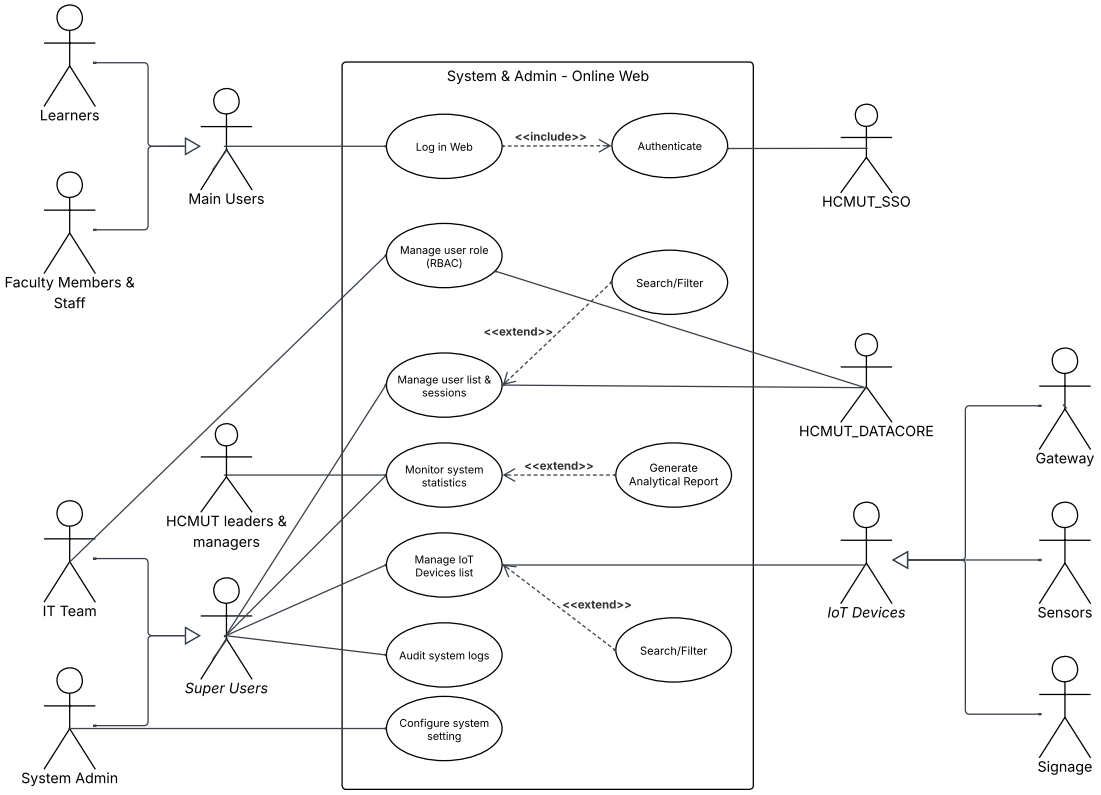

Figure 5: Use case diagram for Module 4.

Table 21: UC 4.1: Authenticate on WEB

Field	Details
Use Case ID	4.1
Use Case Name	Authenticate on WEB
Actors	Primary: Learners, Faculty Members & Staff. Supporting: HCMUT_SSO.

Continued on next page

Table 21: UC 4.1: Authenticate on WEB (Continued)

Description	User's credentials authentication process at the WEB app using the university's centralized Single Sign-On system.
Trigger	User navigates to the login page or attempts to access restricted dashboard features.
Pre-conditions	User is at the authentication page.
Post-conditions	System connects with HCMUT_SSO to authenticate. Valid credentials grant access; otherwise, the request is rejected.
Main Flow	1. User navigates to the authentication page. 2. User inputs credentials information (Username and Password) into the page. 3. The system initiates the authentication process by checking the information with fetched data from HCMUT_SSO. 4. If information is valid, the system provides the user entrance and redirects to the dashboard. 5. If information is NOT valid, the system rejects the request and prompts the user to re-input credentials (back to step 2).
Exception Flows	**E1 - Connection to Authentication Module/Host Lost:** The system displays a failsafe page indicating a lost connection to the pipeline or host. The user must refresh the page to check status. **E2 - Connection to HCMUT_SSO Lost:** The system displays a failsafe page indicating that the SSO service is unreachable. Access is temporarily restricted until the connection is restored.

Table 22: UC 4.2: Manage user list and current sessions

Field	**Details**
Use Case ID	4.2
Use Case Name	Manage user list and current sessions

Continued on next page

Table 22: UC 4.2: Manage user list and current sessions (Continued)

Actors	Primary: System Administrator, IT Team. Supporting: HCMUT_DATACORE.
Description	Manage the university user list, oversee role distribution via RBAC, and monitor all currently active parking sessions in real-time.
Trigger	The Administrator or IT staff accesses the User Management or Session Oversight modules on the Web App.
Pre-conditions	Actor passes authentication check (UC 4.1).
Post-conditions	Actor gains control over user management, active oversight of current sessions, and role distribution.
Main Flow	**1. User Management:** After authentication, the actor navigates to the user management sector. The system synchronizes with HCMUT_DATACORE to fetch and display user information. The actor can search (Name, Card ID, User ID) or filter (Type, Status, Faculty) the list. **2. Oversight of Role Distribution:** The actor navigates to the role distribution sector. The system fetches data from the Role Based Authentication Control (RBAC) module. The actor can search or filter based on roles and status. **3. Oversight of Current Sessions:** The actor navigates to the current sessions sector. The system fetches real-time information on active parking sessions. The actor can search or filter sessions using keys like License Plate, Card ID, or entry time.
Exception Flows	**E1 - Connection to Auth/Host Lost:** Displays a failsafe page. The actor must refresh to check the connection status. **E2 - Connection to HCMUT_DATACORE Lost:** Displays a connection lost message and returns an empty user list. The actor must refresh to retry. **E3 - Connection to RBAC Module Lost:** Displays a connection error and returns an empty role-based information list.

Table 23: UC 4.3: Manage user role (RBAC)

Field	Details
Use Case ID	4.3
Use Case Name	Manage user role (RBAC)
Actors	Primary: IT Team. Supporting: HCMUT_DATACORE.
Description	Manage and distribute roles to specialized users within the smart parking operation pipeline.
Pre-conditions	Actor passes authentication check (UC 4.1).
Post-conditions	Actor gains control over role-based user management.
Main Flow	**Add a new role:** 1. After passing authentication, actor navigates to the RBAC sector. 2. Actor chooses to create a new role and selects a name for that role. 3. Actor inputs the designated user information (Name, ID, etc.). 4. The system checks validity with HCMUT_DATACORE; if valid, the user is designated; if not, an invalid message is shown. **Designate a new user for an existing role:** 1. Actor navigates to the RBAC sector and selects an existing role. 2. Actor updates the designated user information. 3. System checks validity with HCMUT_DATACORE. If valid, it asks for confirmation. If confirmed, the user is updated. If invalid, a message is shown. **Remove an existing role:** 1. Actor selects an existing role to remove. 2. System asks for confirmation. If provided, the role and the designated user are removed; if not, the system returns safely.

Continued on next page

Table 23: UC 4.3: Manage user role (RBAC) (Continued)

Exception Flows	**E1 - Connection to Authentication/Host Lost:** Displays a failsafe page. Actor must refresh to check connection status. **E2 - Connection to HCMUT_DATACORE Lost:** Displays a connection lost message and temporarily disables role assignment capabilities.

Table 24: UC 4.4: Audit system logs

Field	**Details**
Use Case ID	4.4
Use Case Name	Audit system logs
Actors	Primary: System Admin, IT Team.
Description	Oversight of activity loggings and system logging to ensure transparency and track operational/technical issues.
Pre-conditions	Actor passes authentication check (UC 4.1).
Post-conditions	Actor gains oversight over reports and loggings of system and onsite activities.

Continued on next page

Table 24: UC 4.4: Audit system logs (Continued)

Main Flow	1. After passing authentication, the actor navigates to the system log sector. 2. The system fetches logs from the central database and displays them in categorized lists: - **System Log:** Records of system events and operations. - **Fee Policy Log:** Tracks changes to parking fee rules and privileges. - **Onsite Activity Log:** Captures physical parking activities. - **Onsite Error Report:** Logs errors occurring at the physical infrastructure level. - **System Error Report:** Captures backend/software failures. 3. The actor may search for specific entries using keys (Name, ID, Time, Keywords). 4. The actor may filter through the lists using the filter feature to narrow down results.
Alternate Flows	None.
Exception Flows	**E1 - Connection to Authentication/Host Lost:** Displays a failsafe page. The actor must refresh to check the connection status. **E2 - Connection to Central Database Lost:** Displays a connection lost message and returns empty lists for the affected categories.

Table 25: UC 4.5: Monitor system statistics

Field	**Details**
Use Case ID	4.5
Use Case Name	Monitor system statistics
Actors	Primary: System Administrator, IT Team, HCMUT leaders & managers.

Continued on next page

Table 25: UC 4.5: Monitor system statistics (Continued)

Description	This process allows authorized users to view and analyze real-time and historical performance data of the parking facility, including occupancy rates, device health, and usage patterns.
Pre-conditions	1. User has successfully authenticated via HCMUT_SSO. 2. User has the appropriate role-based access permissions (Admin, IT, or Management). 3. The system is receiving data from the IoT gateways and sensors.
Post-conditions	1. The system generates and displays visual reports (charts/graphs) of the requested statistics. 2. All data access is logged for traceability.
Main Flow	1. Actor navigates to the "System Statistics & Dashboard" section of the Web application. 2. System fetches near real-time occupancy data and historical logs. 3. Actor selects the desired reporting cycle (daily, weekly, monthly). 4. System processes the raw data and renders visual statistics on the dashboard. 5. Actor reviews the statistics and may export the report for further auditing or management review.

Continued on next page

Table 25: UC 4.5: Monitor system statistics (Continued)

Exception Flows	**E1: Sensor or Gateway Disconnection:** System detects "intermittent connectivity" or "gateway disconnection." Dashboard displays a warning indicator and uses the last known state. **E2: Data Inconsistency/Sensor Malfunction:** System identifies conflicting data and marks the specific parking slot as "Sensor Malfunction" in the statistics view. **E3: Connection to authentication module is lost:** Display failsafe page indicating the connection lost to authentication pipeline. Actor refreshes page to check connection status. **E4: Connection to Central Database is lost:** Display message indicating the connection lost to Central Database. Returns empty lists on affected information. **E5: Connection to host is lost:** Display failsafe page indicating the connection lost to the host. Actor refreshes page to check connection status.

Table 26: UC 4.6: Manage IoT Devices list

Field	**Details**
Use Case ID	4.6
Use Case Name	Manage IoT Devices list
Actors	Primary: System Administrator, IT Team. Supporting actors: Sensors, Gateway, Signage.
Description	This process allows technical staff to maintain the physical infrastructure of the smart parking system by adding, removing, or modifying the configuration of IoT hardware (sensors, gateways, and signs) within the network.
Trigger	Actor navigates to the "IoT Device Management" portal in the Web application.

Continued on next page

Table 26: UC 4.6: Manage IoT Devices list (Continued)

Pre-conditions	1. User has successfully authenticated and has Administrative or IT Team privileges. 2. The IoT devices are physically installed and connected to the university's local network.
Post-conditions	1. The system is updated with the new device configurations. 2. The central monitoring system begins receiving data from new/modified sensors or sending data to updated signage. 3. An audit log entry is created detailing the change made to the hardware list.
Main Flow	1. Actor navigates to the "IoT Device Management" portal in the Web application. 2. System displays a list of all currently registered Sensors, Gateways, and Signage devices. 3. Actor selects an action: Add (new device), Edit (modify existing configuration), or Remove. 4. System processes the raw data and renders visual statistics on the dashboard. 5. System validates the device connection and saves the configuration. 6. System updates the live monitoring dashboard and signage display logic to include the change.

Continued on next page

Table 26: UC 4.6: Manage IoT Devices list (Continued)

Exception Flows	**E1: Device Not Found:** System fails to ping the new device. Display "Device Not Reachable" error message. Actor checks physical connection. **E2: Unauthorized Modification Attempt:** A user without proper RBAC privileges tries to delete a critical Gateway. System rejects the request and displays "Access Denied: Insufficient Privileges." The incident is logged in the Audit System Logs. **E3: Connection to authentication module is lost:** Display failsafe page indicating the connection lost to authentication pipeline. Actor refreshes page to check connection status. **E4: Connection to Central Database is lost:** Display message indication the connection lost to Central Database. Returns empty lists on affected information. Actor refreshes page to check connection status. **E5: Connection to host is lost:** Display failsafe page indicating the connection lost to the host. Actor refreshes page to check connection status.

Table 27: UC 4.7: Configure System settings

Field	**Details**
Use Case ID	4.7
Use Case Name	Configure System settings
Actors	Primary: System Admin.
Description	This process allows the authorized System Administrator to adjust global parameters and operational thresholds that govern system behavior.
Trigger	The Administrator navigates to the "System Configuration & Settings" portal.

Continued on next page

Table 27: UC 4.7: Configure System settings (Continued)

Pre-conditions	System Administrator has successfully authenticated via HCMUT_SSO.
Post-conditions	1. New parameters are applied system-wide in near real-time. 2. Changes are recorded in the audit logs for traceability. 3. Connected IoT devices (like signage) update their logic based on the new thresholds.
Main Flow	1. Administrator navigates to the "System Configuration & Settings" portal. 2. System displays a menu of adjustable parameters. 3. Administrator inputs new values or selects options from a dropdown menu. 4. Administrator clicks "Apply & Save."
Exception Flows	**E1: Invalid Parameter Input:** Administrator enters a non-logical value. System displays an error message: "Invalid Input: Please enter a value between 0-100." **E2: Unauthorized Access:** A user with lower-level permissions (Like IT Team) attempts to access the page. System prevents access and displays: "Access Denied: Administrative Clearance Required". The attempt is logged in the Audit Logs. **E3: Connection to authentication module is lost:** Display failsafe page indicating the connection lost to authentication pipeline. Actor refreshes page to check connection status. **E4: Connection to host is lost:** Display failsafe page indicating the connection lost to the host. Actor refreshes page to check connection status.